MARKOV DECISION PROCESSES + REINFORCEMENT LEARNING

From classical MDP theory to Q-learning, real-world control, and Deep RL

AI Fundamentals

Lecture roadmap

A coherent path from decision problems to learning agents

Why sequential decisions?

Immediate actions change future opportunities.

MDP model

States, actions, transitions, rewards, discounting.

Value & Bellman

How future consequences are summarized mathematically.

Planning

Value iteration and policy iteration when the model is known.

Reinforcement learning

Learning from interaction when the model is unknown.

Q-learning & beyond

TD learning, SARSA, DQN, Actor–Critic and applications.

Why sequential decision-making?

The action that looks best now may be poor in the long run

- A decision changes the next state of the environment.
- The next state changes which actions and rewards are available later.
- Uncertainty means the same action may produce different outcomes.
- We therefore optimize expected long-term return—not only immediate reward.

Example: industrial machine

CONTINUE gives +10 production now, but repeated operation may increase temperature and failure risk. MAINTENANCE costs −15 now, yet can prevent a future −200 failure.

KEY IDEA **Sequential decision-making connects present choices to future consequences.**

Search vs. MDP planning

Why a path is not enough under uncertainty

Classical search

Often seeks a sequence of actions: START → RIGHT → RIGHT → UP → GOAL. This works when action outcomes are sufficiently predictable.

MDP planning

Produces a policy: what to do from each possible state. If RIGHT slips to DOWN, the policy still tells the agent what to do next.

KEY IDEA Search often returns a path; an MDP solution is a policy.

Markov Decision Processes

A mathematical model for sequential decision-making under uncertainty

An MDP is defined by:

$$\mathcal{M} = (\mathcal{S}, \mathcal{A}, \mathcal{P}, \mathcal{R}, \gamma)$$

1. **State space $\mathcal{S}$**
   A set of states $s \in \mathcal{S}$ representing possible situations the agent can be in.

2. **Action space $\mathcal{A}$**
   A set of actions $a \in \mathcal{A}$ the agent can choose from.

3. **Transition model $\mathcal{P}$**
   $\mathcal{P}(s' \mid s, a)$: probability that action a in state s leads to next state s'. Also called the dynamics.

4. **Reward function $\mathcal{R}$**
   $\mathcal{R}(s, a, s')$: immediate scalar reward after the transition. Sometimes written as $\mathcal{R}(s)$ or $\mathcal{R}(s')$.

5. **Discount factor γ**
   $\gamma \in [0, 1]$: determines how much future rewards matter.

Gridworld MDP Example

An agent moves on a grid with **stochastic actions**, walls and terminal rewards.



Stochastic transition model

Intended direction succeeds with 80%, left and right directions with 10% each.



Agent–environment interaction

One cycle is the basic unit of experience



- The agent controls its action—not necessarily the outcome.
- The environment determines the next state and reward according to its dynamics.
- Repeated cycles create a trajectory of experience.

KEY IDEA RL data naturally arrives as transitions: (s, a, r, s′).

State: what does the agent know?

A state should summarize information relevant to future decisions

Gridworld

State can be position: s = (row, column). Small, discrete, easy to tabulate.

Battery

s = (SOC, price, demand, solar, hour). Operational context matters.

Industrial motor

s = (temperature, vibration, load, health). Sensor variables describe condition.

KEY IDEA **State design determines what distinctions the agent is able to learn.**

MARKOV DECISION PROCESSES

The Markov Property

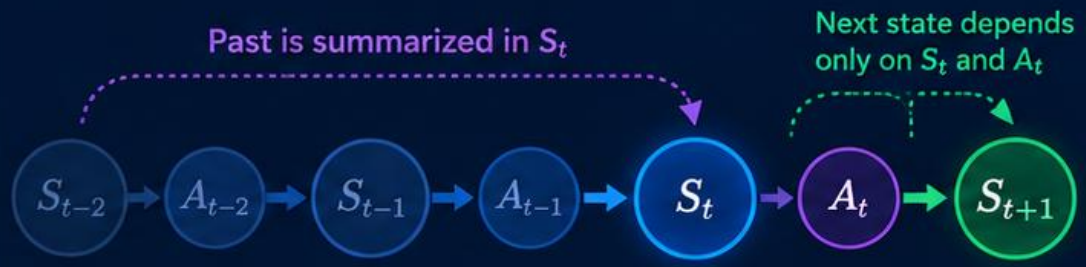
Relevant history is summarized in the current state

Formal definition

$$P(\underbrace{S_{t+1} \mid S_t, A_t, S_{t-1}, A_{t-1}, \dots}_{\text{Whole history}}) = P(\underbrace{S_{t+1} \mid S_t, A_t}_{\text{Only current state and action needed}})$$

Intuition

If the current state contains all information relevant for predicting the future, then older history does not provide additional predictive information.



Bad state design

State contains insufficient information, so the future cannot be predicted from the current state alone.

State:
position = **100 m**
Speed: **0 km/h**

State:
position = **100 m**
Speed: **120 km/h**



Both situations look identical if the state is only position = 100 m, but their future (after braking) will be **very different**.

Better state design

Include all variables relevant for predicting future outcomes.

State:
position = **100 m**
speed = **0 km/h**
direction = →

State:
position = **100 m**
speed = **120 km/h**
direction = →



Now the two situations are **distinguished**, and the next state can be **predicted** from the current state and action.

KEY IDEA | The Markov property is partly a property of **how we represent the state.**

Actions

The choices available to the agent

Discrete

Gridworld: UP, DOWN, LEFT, RIGHT
Battery: CHARGE, HOLD, DISCHARGE

Continuous

Steering angle, motor torque, valve opening, charging power.

State-dependent

Some actions may be invalid: cannot discharge an empty battery or move through a wall.

KEY IDEA **Action-space structure influences which RL algorithms are practical.**

MARKOV DECISION PROCESSES

Transition Model

Describes how the environment responds to an action

Definition

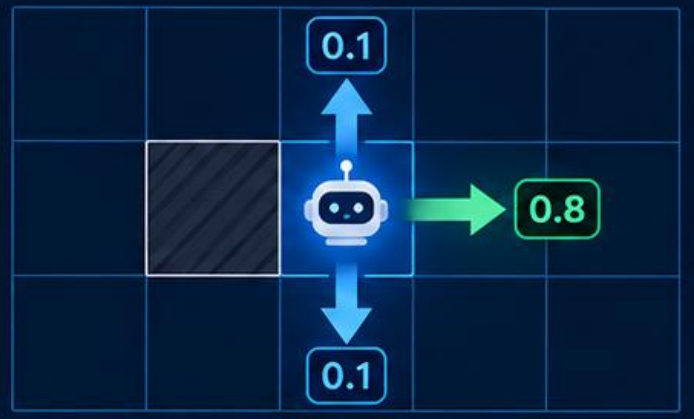
The **transition model** specifies the probability of the next state when the agent takes action a in state s:

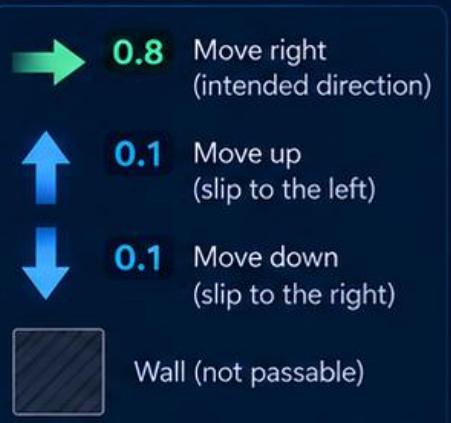
$$P(s' \mid s, a)$$

- s – current state
- a – action taken
- s' – next state
- $P(s' \mid s, a)$ – probability of transitioning to s'

Gridworld example (stochastic transitions)

The agent intends to move RIGHT. However, due to noise, it may move in a different direction.





Deterministic transition model

One action leads to exactly one next state.

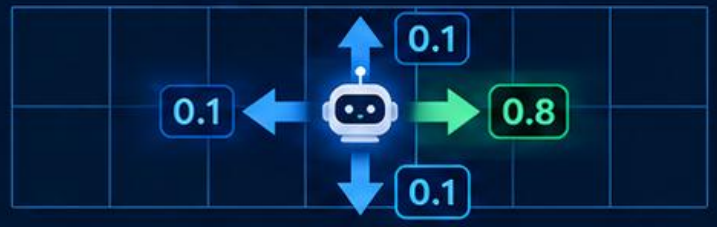
$P(s' \mid s, a) = 1$ for one s', 0 otherwise.



Stochastic transition model

One action leads to a distribution over next states.

$$\sum_{s'} P(s' \mid s, a) = 1$$



Transition probabilities for an example state

From state $s = (2, 2)$ with action RIGHT:

Next state s'	Probability $P(s' \mid s, \text{RIGHT})$
(2, 3) (right)	0.8
(1, 2) (up)	0.1
(3, 2) (down)	0.1
(2, 1) (left, wall)	0.0

KEY IDEA | The transition model captures **uncertainty** in action outcomes. It defines a **probability distribution** over next states.

Stochastic transition in Python

Simulating uncertain movement

```
import random

def move_right():
    x = random.random()
    if x < 0.8:
        return "RIGHT"
    elif x < 0.9:
        return "UP"
    return "DOWN"

for _ in range(10):
    print(move_right())
```

How to read it

random.random() returns a value in [0,1). Intervals implement probabilities: 0–0.8 → RIGHT, 0.8–0.9 → UP, 0.9–1.0 → DOWN. Repeated calls reveal stochastic outcomes.

KEY IDEA **Simulation lets us create experience even when outcomes are probabilistic.**

Expected value

Compare uncertain outcomes using probability-weighted consequences

$$E[R] = \Sigma\, p_i\, r_i$$

Expected reward is not the most likely reward—it averages all possible outcomes by probability.

Example

90% chance of +10
10% chance of −50

E[R] = 0.9(10) + 0.1(−50) = 4

```
expected_reward = 0.9 * 10 + 0.1 * (-50)
print(expected_reward)  # 4.0
```

KEY IDEA **Uncertainty is handled by expectations, not by pretending one outcome is guaranteed.**

Rewards

Immediate feedback about an outcome

Gridworld

Goal +10
Danger −10
Normal movement −1

The step cost encourages shorter routes.

Warehouse robot

Delivery +100
Collision −200
Move −1
Wait −2

Industrial control

Production +10
Reduced load +4
Maintenance −15
Failure −200

KEY IDEA **Reward encodes the immediate objective signal—but not necessarily all safety constraints.**

Reward ≠ value

Immediate feedback and long-term desirability are different

Reward

What happened immediately after the action? A state near the goal may have reward 0 right now.

Value

How much future return is expected from here? A zero-reward state can be very valuable if it leads to +100 next.

KEY IDEA Reward is local; value summarizes the future.

Return

The quantity an RL agent ultimately tries to maximize

$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \dots$$

Discounted cumulative future reward

Example

Rewards: 5, 10, 20
$\gamma = 0.9$
$G = 5 + 0.9(10) + 0.9^2(20) = 30.2$

```
rewards = [5, 10, 20]
gamma = 0.9
G = sum(gamma**t * r
        for t, r in enumerate(rewards))
print(G)  # 30.2
```

KEY IDEA **Return turns a stream of future rewards into one decision objective.**

Discount factor γ

How strongly do future rewards matter?

γ near 0

Strong emphasis on near-term reward. Distant consequences contribute very little.

γ near 1

Future outcomes remain important over a longer horizon.

Mathematical role

Discounting can keep infinite-horizon returns finite and makes Bellman operators well behaved.

KEY IDEA γ is a modeling choice about the effective decision horizon—not simply “patience.”

Discounting: numerical intuition

A reward 10 steps away can matter very differently

Discounted contribution = $\gamma^{10} \times 100$

$\gamma = 0.50$

≈ 0.10

$\gamma = 0.80$

≈ 10.74

$\gamma = 0.90$

≈ 34.87

$\gamma = 0.99$

≈ 90.44

KEY IDEA **The same future reward can be almost irrelevant or highly influential depending on γ.**

Policy

A strategy for acting in every state

$$\pi(a \mid s)$$

Probability of choosing action a in state s

Deterministic policy

$\pi(s) = a$
Example: state B → RIGHT. One prescribed action for each state.

Stochastic policy

$\pi(\text{RIGHT}|B)=0.8$, $\pi(\text{UP}|B)=0.2$. The policy itself can randomize.

KEY IDEA A policy is the object we ultimately want to improve or learn.

Real-world policy: battery control

A policy maps operating conditions to decisions

Low price + low SOC

CHARGE
Store inexpensive energy for later.

High price + high SOC

DISCHARGE
Use stored energy when grid energy is expensive.

Intermediate conditions

HOLD or partial action depending on forecasts, constraints and future value.

KEY IDEA A learned policy may resemble sensible rules without those rules being hand-coded.

State-value function Vπ(s)

How good is it to be in this state under policy π?

$$V^{\pi}(s) = E_{\pi}[\, G_t \mid S_t = s \,]$$

Expected discounted return from state s while following policy π



KEY IDEA V evaluates states under a specified policy.

Action-value function Qπ(s,a)

How good is this action in this state?

$$Q^{\pi}(s,a) = E_{\pi}[\, G_t \mid S_t = s,\ A_t = a \,]$$

Question answered

"If I take RIGHT now from state A, then continue according to π, what return should I expect?"

Why useful

Q-values directly compare actions and therefore make action selection easy.

KEY IDEA V rates states; Q rates state–action pairs.

V and Q together

Optimal state value is the best available action value

$$V^*(s) = \max_a Q^*(s,a)$$

At state A

Q(A,LEFT)=2
Q(A,RIGHT)=8
Q(A,UP)=4
Q(A,DOWN)=−5

Therefore

V*(A)=8
Optimal action = RIGHT

KEY IDEA Q-values contain both evaluation and an immediate recipe for choosing actions.

Bellman principle

Break a long future into “now + later”

Current value = Immediate reward + Discounted future value

- The future after one transition is itself another decision problem.
- This recursive structure lets us avoid enumerating an entire infinite future at once.
- Bellman equations formalize this recursion.

KEY IDEA **Bellman recursion is the mathematical bridge between present and future.**

Simple Bellman backup

Value propagates backward from desirable states

$$V(B) = -1 + 0.9 \times 10 = 8$$



$$\text{Then } V(A) = -1 + 0.9 \times 8 = 6.2$$

KEY IDEA A state becomes valuable because it leads to valuable future states.

MARKOV DECISION PROCESSES

The Bellman Principle

A recursive relationship between the value of a state and the values of its successors

Bellman expectation equation (for a fixed policy π)

$$V^{\pi}(s) = \sum_{a} \pi(a \mid s) \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^{\pi}(s') \right]$$

- $\pi(a \mid s)$ — probability of taking action a in state s
- $P(s' \mid s, a)$ — probability of next state s'
- $R(s, a, s')$ — immediate reward
- $V^{\pi}(s')$ — expected future value from the next state

Gridworld example: Bellman expectation

For a simple policy (e.g., random actions), the value of a state is the expected reward plus the discounted value of successor states.



Value of state s under random policy

$$V(s) = 0.25 \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V(s') \right]$$

Intuition: value now = immediate reward + discounted future value



Bellman optimality equation

For the optimal value function $V^{*}(s)$:

$$V^{*}(s) = \max_{a \in A} \sum_{s'} P(s' \mid s, a) \left[R(s, a, s') + \gamma V^{*}(s') \right]$$

The optimal value of a state is the maximum expected return over all possible actions.

KEY IDEA | The Bellman principle expresses the value of a state in terms of **immediate reward** and the **values of successor states.**

Bellman optimality equation

Replace policy averaging with choosing the best action

$$V^*(s) = \max_a \Sigma_{s'} P(s'|s,a)[R + \gamma V^*(s')]$$

Evaluate each action → Average its possible outcomes → Add future value → Choose max

KEY IDEA Optimal control adds a max over actions to Bellman reasoning.

Stochastic Bellman example

Expected future value changes action quality

RIGHT action

80% → GOOD with V=10
20% → BAD with V=−5
Step reward = −1
γ = 0.9

Calculation

Q(s,RIGHT)
= 0.8[−1+0.9(10)] + 0.2[−1+0.9(−5)]
= 6.4 − 1.1
= 5.3

KEY IDEA **The best action depends on probability-weighted long-term consequences.**

Planning vs. learning

The key fork in the lecture

Planning: model known

We know or can query P and R. Use dynamic programming methods such as value iteration and policy iteration.

Learning: model unknown

We observe transitions and rewards from experience. Use RL to estimate values, policies, or models.

KEY IDEA **MDP is the problem framework; planning and RL are different ways to solve it.**

Value Iteration

Repeated Bellman optimality backups

$$V_{k+1}(s) = \max_a \Sigma_{s'} P(s'|s,a)[R + \gamma V_k(s')]$$

Initialize V_0 → Bellman backup → Update all states → Repeat until stable

KEY IDEA Value iteration directly pushes values toward the optimal fixed point.

Value Iteration intuition

Goal information propagates backward



- Each sweep uses values from the previous estimate.
- States one step from reward improve first.
- Then their predecessors improve.
- Eventually values stabilize within a tolerance.

KEY IDEA Dynamic programming turns local Bellman backups into global long-term values.

Value Iteration in Python

A minimal deterministic chain

```
import numpy as np

gamma = 0.9
V = np.zeros(4)
goal_value = 10

for k in range(5):
    new_V = V.copy()
    new_V[2] = -1 + gamma * goal_value
    new_V[1] = -1 + gamma * V[2]
    new_V[0] = -1 + gamma * V[1]
    V = new_V
    print(k, np.round(V, 2))
```

What the code demonstrates

The values are not rewards. They are estimates of future return. Repeated updates move information from the goal backward to earlier states.

KEY IDEA **Python makes the Bellman recursion visible iteration by iteration.**

Policy Iteration

Evaluate a policy, then improve it



Policy evaluation

Compute Vπ: how good are states if we continue following the current policy?

Policy improvement

For each state, choose the action with the highest one-step lookahead value.

KEY IDEA **Policy iteration alternates between understanding a policy and making it better.**

When the model is unknown

Why classical planning becomes difficult

- Real systems may not provide exact transition probabilities.
- Rewards may depend on complex physical or economic effects.
- Dynamics can be too expensive or difficult to model accurately.
- We may instead have interaction data or a simulator.

Example

What is P(failure tomorrow | vibration=6.5, load=92%, CONTINUE)? In many plants, we do not know this exact probability.

KEY IDEA **Reinforcement learning replaces complete prior knowledge with experience-driven estimation.**

Reinforcement Learning

Learning a decision strategy from interaction



- No label tells the agent “the correct action.”
- Feedback can be delayed: an action may matter many steps later.
- The agent’s behavior affects which data it observes.

KEY IDEA RL is learning under feedback, delay, and sequential dependence.

REINFORCEMENT LEARNING

What is Reinforcement Learning?

Learning to make good decisions through interaction with the environment

Goal: learn a policy that maximizes expected long-term reward **without knowing the environment model.**

The RL interaction loop

The agent learns from experience (trial and error).



Reinforcement Learning vs Supervised Learning

Supervised Learning

Learn from labeled examples (corresponding inputs and targets).



Reinforcement Learning

Learn from interaction and reward (no correct action is given).



Experience tuple

Each step generates one transition.

S_t state → A_t action → R_{t+1} reward → S_{t+1} next state

$$(S_t, A_t, R_{t+1}, S_{t+1})$$

A sequence of transitions forms a trajectory (episode).

Key RL formulation

Policy
Strategy for choosing actions.
$$\pi(a \mid s)$$

Objective
Maximize expected return.
$$J(\pi) = \mathrm{E}_\pi[G_t]$$

Return
Sum of discounted future rewards.
$$G_t = R_{t+1} + \gamma R_{t+2} + \gamma^2 R_{t+3} + \cdots$$

Learning through exploration and exploitation

The agent must try new actions while also using what it has learned.

Exploration
Try uncertain actions to discover better outcomes.



Balance for better long-term performance

Exploitation
Choose the best-known action according to the current policy or Q-values.



Simple RL example — Gridworld

The agent learns to reach the goal using Q-learning.



Early training (mostly random) → Middle training (partial knowledge) → Learned policy (near optimal)

Q-learning update rule

Update the value of the action taken using the observed reward and the best future value.

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- α – learning rate (how fast to update)
- γ – discount factor (importance of future)
- r – observed reward
- $\max_{a'} Q(s',a')$ – best estimated future value

Q-learning in Python (simplified)

```
# Update Q-value after observing (s, a, r, s')
best_future = np.maxQ[next_state])
Q[state, action] += alpha * (reward + gamma * best_future - Q[state, action])
```

RL vs supervised learning

Why RL is not just classification

Supervised learning

Input → target label
The dataset usually provides the desired answer for each example. Samples are often treated as independent.

Reinforcement learning

State → action → consequence → reward
The agent must discover useful actions; decisions change future data.

KEY IDEA RL learns from consequences rather than direct action labels.

Trajectories and episodes

Experience has temporal structure

$$S_0, A_0, R_1, S_1, A_1, R_2, S_2, \ldots$$

A trajectory is an ordered sequence of interactions.

Episodic task

A game, delivery, or maze ends at a terminal state. Training uses many episodes.

Continuing task

HVAC, process control, or grid operation may continue indefinitely; discounting/average reward handles the horizon.

KEY IDEA **Temporal order matters because earlier actions influence later states and rewards.**

Exploration vs exploitation

Learning requires information as well as reward

Exploit

Choose the action currently believed to be best. Good for short-term performance.

Explore

Try uncertain alternatives to discover whether current beliefs are wrong or incomplete.

KEY IDEA **Too little exploration can lock the agent into a mediocre policy; too much wastes reward.**

ε-greedy exploration

A simple practical compromise

with probability ε: random action | otherwise: argmax_a Q(s,a)

```
def choose_action(Q, state, epsilon=0.1):
    if random.random() < epsilon:
        return random.randrange(Q.shape[1])
    return np.argmax(Q[state])
```

Interpretation

ε=0.1 means roughly 10% exploratory actions and 90% greedy actions. ε is often decayed during training.

KEY IDEA Exploration is a data-collection strategy, not random noise for its own sake.

Monte Carlo idea

Learn from complete returns after an episode

$$V(s) \leftarrow V(s) + \alpha \left[G_t - V(s) \right]$$

- Wait until enough future rewards are observed to compute the return G_t.
- Compare the observed return with the current estimate.
- Move the estimate toward what actually happened.

KEY IDEA **Monte Carlo uses sampled returns; TD methods learn before the episode is finished.**

REINFORCEMENT LEARNING

Monte Carlo (MC) Idea

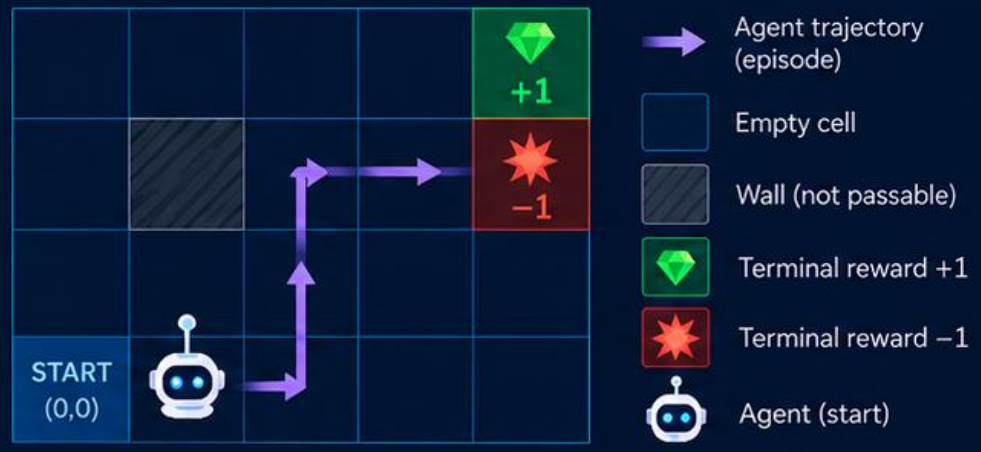
Learn from complete episodes by averaging the observed returns

Key intuition

Monte Carlo methods estimate the value of a state or a state–action pair by averaging the **actual returns** after many **complete episodes, without using a model or bootstrapping.**

1 Generate complete episodes

Follow a policy π to interact with the environment until a terminal state is reached.



2 Observe the episode and rewards

An episode is a sequence of states, actions and rewards ending in a terminal state.

t	State s_t	Action a_t	Reward r_{t+1}
0	(0,0)	→	0
1	(0,1)	→	0
2	(0,2)	↑	0
3	(1,2)	↑	0
4	(2,2)	→	0
5	(2,3)	→	+1

Episode: $(s_0, a_0, r_1, s_1, \ldots, s_5, r_6 = +1)$

3 Compute returns

For each time step, compute the return (discounted sum of future rewards).

$$G_t = \sum_{k=0}^{T-t-1} \gamma^k R_{t+1+k}$$

In this episode:
$R_1 = R_2 = R_3 = R_4 = R_5 = 0$
$R_6 = +1$
(s_5 is terminal)

t	Reward r_{t+1}	Return G_t ($\gamma = 0.9$)
0	0	$0.9^5 = 0.5905$
1	0	$0.9^4 = 0.6561$
2	0	$0.9^3 = 0.7290$
3	0	$0.9^2 = 0.8100$
4	0	$0.9^1 = 0.9000$
5	+1	1.0000

4 Update value estimates by averaging

For every state (or state–action pair) visited in the episode, update its value estimate using the observed return.

First-visit MC (state value)

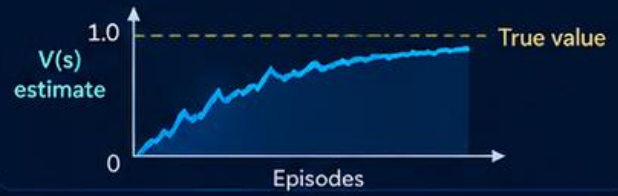
$$V(s) \leftarrow V(s) + \frac{1}{N(s)}\left[G_t - V(s)\right]$$

First-visit MC (action value)

$$Q(s,a) \leftarrow Q(s,a) + \frac{1}{N(s,a)}\left[G_t - Q(s,a)\right]$$

5 Repeat over many episodes

Run many episodes and average the returns. The estimates converge to the true values (as episode count increases).



Important characteristics

- **Model-free**: no knowledge of transition probabilities or reward function is needed.
- Uses **complete episodes** (no bootstrapping).
- Works for episodic tasks (and also continuing tasks with suitable adaptations).
- Provides **unbiased estimates** of value functions in the limit of many episodes.

KEY IDEA | Monte Carlo methods learn the value of states or state–action pairs by **averaging the actual returns** observed after complete episodes, **without using a model** and **without bootstrapping.**

Temporal-Difference learning

Learn from one step plus a current estimate

$$\delta_t = R_{t+1} + \gamma V(S_{t+1}) - V(S_t)$$

δ is the temporal-difference prediction error

δ > 0

Outcome was better than expected → increase the current estimate.

δ < 0

Outcome was worse than expected → decrease the current estimate.

KEY IDEA TD learning updates predictions using a reward plus another prediction: bootstrapping.

TD numerical example

Turning surprise into learning

Observed transition

V(A)=5
Reward=2
V(B)=8
γ=0.9

Calculation

Target = 2 + 0.9×8 = 9.2
TD error = 9.2 − 5 = 4.2
Positive error → A was better than predicted.

KEY IDEA **Prediction error is the learning signal connecting experience to updated value estimates.**

Q-learning

Model-free learning of optimal action values

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma \max_{a'} Q(s',a') - Q(s,a)]$$



KEY IDEA Q-learning learns the value of acting now plus behaving optimally afterward.

Q-learning equation

Do not memorize symbols without meaning

Current belief

$Q(s,a)$
"What I currently think this action is worth."

New evidence

$r + \gamma \max Q(s',a')$
"What this transition suggests the action is worth."

Correction

$\alpha \times (\text{target} - \text{current})$
"Move partway toward the new evidence."

KEY IDEA Q-learning is incremental error correction.

REINFORCEMENT LEARNING

Q-learning

Model-free learning of the optimal action-value function

Goal

Learn the optimal action-value function $Q^*(s,a)$ and an optimal policy $\pi^*(s)$ **without knowing the transition model or reward function.**

Core idea

Learn from interaction using the observed transition (s, a, r, s') and update the Q-value towards a better estimate of the expected return.



Q-learning update rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- $Q(s,a)$: Current Q-value
- α: Learning rate $(0 < \alpha \leq 1)$
- r: Observed reward
- γ: Discount factor $(0 \leq \gamma < 1)$
- $\max_{a'} Q(s',a')$: Best next action value (estimate of future)
- $r + \gamma \max_{a'} Q(s',a') - Q(s,a)$: TD error (target − current)

Intuition

1. Take an action a in state s (using an exploration strategy, e.g. ε-greedy).
2. Observe reward r and next state s'.
3. Update $Q(s,a)$ towards the target $r + \gamma \max_{a'} Q(s',a')$.
4. Repeat many times. Q-values converge to the optimal $Q^*(s,a)$ under suitable conditions.

Example: Gridworld step by step

A simple 3×4 gridworld with noisy-free moves, goal $+1$ and a pit -1.



Sample update

From state $(1,2)$, take action RIGHT and reach $(1,3)$ with reward 0. Assume $\alpha = 0.5$, $\gamma = 0.9$.

1. Old Q-value: $Q((1,2), RIGHT) = 0$
2. Reward: $r = 0$
3. Next state best value: $\max_{a'} Q((1,3), a') = 0.8$ (from current table)
4. Update: $Q_{new} = 0 + 0.5\,[0 + 0.9 \times 0.8 - 0] = 0.36$

Q-table (partial)

State	UP	RIGHT	DOWN	LEFT
(1,1)	0	0	0	0
(1,2)	0	0.36	0	0
(1,3)	0	0	0	0
(2,3)	0.72	0	0	0
...	...	...	...	...

Q-learning algorithm (Python)

```python
import numpy as np

Q = np.zeros((n_states, n_actions))
alpha = 0.5      # learning rate
gamma = 0.9      # discount factor
epsilon = 0.1    # exploration rate

for episode in range(num_episodes):
    s = env.reset()
    done = False
    while not done:
        if np.random.rand() < epsilon:
            a = env.action_space.sample() # explore
        else:
            a = np.argmax(Q[s])           # exploit

        s_next, r, done, _ = env.step(a)
        best_next = np.max(Q[s_next])
        Q[s, a] += alpha * (r + gamma * best_next - Q[s,a])
        s = s_next
```

Exploration vs Exploitation

Exploration

- Try uncertain actions to discover better rewards.
- Common strategies: ε-greedy, softmax, or optimistic initialization.

Balance for better long-term performance

Exploitation

- Choose the action with the highest current Q-value.
- Needed to achieve good performance once good estimates are learned.

Key properties

Model-free: No need to know $P(s'|s,a)$ or $R(s,a)$.

Converges to Q^*: With sufficient exploration and suitable learning rate.

Off-policy: Learns the optimal policy even while following an exploratory policy (e.g. ε-greedy).

Q-learning: complete numerical update

One experience, one changed belief

Given

$Q(A,\text{RIGHT})=4$
$r=2$
$\max Q(B,\cdot)=8$
$\gamma=0.9$
$\alpha=0.5$

Compute

Target = $2 + 0.9\times8 = 9.2$
Error = $9.2 - 4 = 5.2$
New Q = $4 + 0.5\times5.2 = 6.6$

KEY IDEA **The action becomes more valuable because experience was better than expected.**

Q-learning in Python: one update

The code mirrors the mathematics

```
current_q = 4.0
reward = 2.0
best_future_q = 8.0
alpha = 0.5
gamma = 0.9

target = reward + gamma * best_future_q
td_error = target - current_q
new_q = current_q + alpha * td_error

print(target, td_error, new_q)
```

Result

Target = 9.2
TD error = 5.2
New Q = 6.6

Each line corresponds to one term of the update equation.

KEY IDEA **Students should be able to map every Python variable back to the equation.**

Learning rate α

How much should one experience change the estimate?

Small α

Slow, stable updates. Old experience remains influential. Useful in noisy settings but may learn slowly.

Large α

Fast adaptation. New samples dominate more strongly, but estimates can become noisy or unstable.

KEY IDEA **α controls update size; γ controls future weighting; ε controls exploration.**

Three parameters students must separate

α, γ and ε solve different problems

α — learning rate

How strongly should new experience update current knowledge?

γ — discount factor

How strongly should future rewards influence present value?

ε — exploration rate

How often should we try alternatives rather than act greedily?

KEY IDEA **Learning, horizon, and exploration are conceptually distinct.**

Complete Q-learning environment

A five-state chain



Actions: LEFT / RIGHT • Step reward: −1 • Goal reward: +10

Learning objective

Discover that repeatedly choosing RIGHT reaches the goal efficiently—even though the Q-table begins with no knowledge.

KEY IDEA A tiny environment lets us inspect every part of the learning process.

Q-table initialization

Knowledge starts neutral

```
import numpy as np

N_STATES = 5
N_ACTIONS = 2
Q = np.zeros((N_STATES, N_ACTIONS))

# column 0 = LEFT
# column 1 = RIGHT
print(Q)
```

Interpretation

Rows represent states. Columns represent actions. Initially all actions have equal estimated value: 0. Experience will break the symmetry.

KEY IDEA Tabular RL explicitly stores one value per state–action pair.

Environment step function

Translate an action into next state and reward

```
LEFT, RIGHT = 0, 1
GOAL = 4

def step(state, action):
    if action == LEFT:
        next_state = max(0, state - 1)
    else:
        next_state = min(GOAL, state + 1)

    reward = 10 if next_state == GOAL else -1
    return next_state, reward
```

Purpose

This function is the environment dynamics. It enforces boundaries, moves the agent, and produces the reward used for learning.

KEY IDEA An RL environment needs a clear transition and reward interface.

Training loop

The core algorithm in executable form

```
for episode in range(1000):
    state = 0
    while state != GOAL:
        action = choose_action(Q, state, epsilon)
        next_state, reward = step(state, action)

        target = reward + gamma * np.max(Q[next_state])
        Q[state, action] += alpha * (target - Q[state,
action])

        state = next_state
```

Cycle

1 Choose action
2 Interact
3 Observe reward + next state
4 Compute target
5 Update Q
6 Move to next state
7 Repeat

KEY IDEA The entire Q-learning algorithm is a repeated state–action–reward–update cycle.

What learning looks like

Behavior improves as reward information propagates

Early episode

0 → 1 → 0 → 1 → 2 → 1 → 2 → 3 → 4

Exploration and weak Q-values create inefficient movement.

Later episode

0 → 1 → 2 → 3 → 4

Q-values now favor RIGHT from each non-terminal state.

KEY IDEA **Learning is visible as both changing Q-values and improving trajectories.**

Extracting the learned policy

Turn Q-values into decisions

$$\pi(s) = \text{argmax}_a \, Q(s,a)$$

```
action_names = ["LEFT", "RIGHT"]

for state in range(GOAL):
    best_action = np.argmax(Q[state])
    print(state, action_names[best_action])
```

Expected result

State 0 → RIGHT
State 1 → RIGHT
State 2 → RIGHT
State 3 → RIGHT

KEY IDEA Learning Q is useful because a greedy policy can be read directly from the table.

SARSA

An on-policy TD control algorithm

$$Q(s,a) \leftarrow Q(s,a) + \alpha[r + \gamma Q(s',a') - Q(s,a)]$$

Q-learning

Uses max Q(s′,a′): learns toward the greedy target policy even while behavior explores.

SARSA

Uses the actual next action a′ chosen by the behavior policy. Learning reflects exploratory behavior.

KEY IDEA A small target difference creates the classic off-policy vs on-policy distinction.

REINFORCEMENT LEARNING

SARSA

On-policy Temporal-Difference Control

Key idea

SARSA is an **on-policy** TD control algorithm that learns the action-value function using the **actual next action** taken by the current policy.

1. What is SARSA?

- **On-policy learning:** learns the value of the current policy (the one being followed).
- Uses the tuple (s, a, r, s', a'), hence the name SARSA.
- Updates $Q(s,a)$ towards the value of the next action **actually taken** a', not the best possible action.

2. SARSA update rule

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma\, Q(s',a') - Q(s,a) \right]$$

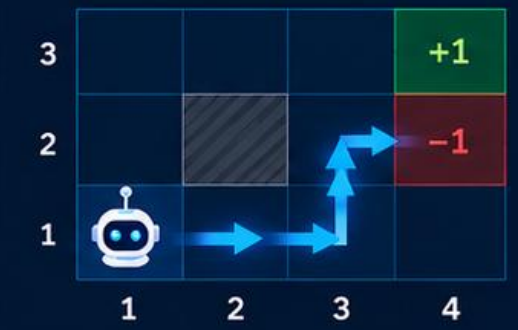
- $Q(s,a)$: Current Q-value
- α: Learning rate $(0 < \alpha \leq 1)$
- r: Observed reward
- γ: Discount factor $(0 \leq \gamma < 1)$
- $Q(s',a')$: Q-value of next action actually taken
- $r + \gamma Q(s',a') - Q(s,a)$: TD error (target − current)

3. Intuition

1. Take action a in state s using current policy π (e.g. ε-greedy).
2. Observe reward r and next state s'.
3. Choose next action a' using the **same current policy** π.
4. Update $Q(s,a)$ towards $r + \gamma Q(s',a')$.

4. Gridworld example

A simple 4×3 gridworld with noisy moves.



5. Step-by-step SARSA update

Suppose the agent follows an ε-greedy policy, with $\alpha = 0.5$, $\gamma = 0.9$.

Step	s	a	r	s'	a' (chosen)	Current Q(s, a)	Target $r + \gamma Q(s',a')$	Updated $Q(s,a)$
1	(1,1)	Right	0	(1,2)	Right	0.00	$0 + 0.9 \cdot 0 = 0$	0.00
2	(1,2)	Right	0	(1,3)	Up	0.00	$0 + 0.9 \cdot 0 = 0$	0.00
3	(1,3)	Up	0	(2,3)	Up	0.00	$0 + 0.9 \cdot 0 = 0$	0.00
4	(2,3)	Up	+1	(3,3)	(terminal)	0.00	$1 + 0.9 \cdot 0 = 1.00$	0.50

6. SARSA vs Q-learning

SARSA (on-policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma Q(s',a') - Q(s,a) \right]$$

- Uses the next action a' actually taken by the policy.
- Learns the value of the current (possibly exploratory) policy.
- Tends to be more conservative (safer) because it accounts for exploratory actions.
- Often preferred in stochastic environments where safety matters (e.g. robotics).

Q-learning (off-policy)

$$Q(s,a) \leftarrow Q(s,a) + \alpha \left[r + \gamma \max_{a'} Q(s',a') - Q(s,a) \right]$$

- Uses the best possible next action (max) regardless of the action actually taken.
- Learns the optimal policy even while following an exploratory policy.
- Usually learns faster but can be riskier during learning.
- More aggressive, suitable when exploration risk is low (e.g. simulation, games).

KEY TAKEAWAY | SARSA updates Q-values using the **next action actually taken**, making it an **on-policy method** that learns the value of the current policy.

Real-world example: battery energy management

A practical MDP/RL mapping

STATE

SOC
Electricity price
Demand
Solar output
Hour / forecast

ACTION

Charge
Hold
Discharge
(or continuous power)

REWARD

− energy cost
− degradation
− constraint penalties

POLICY

When and how strongly to charge or discharge.

KEY IDEA The same MDP vocabulary maps directly onto an energy-control problem.

Battery: why state design matters

SOC alone cannot explain the best action

State = SOC only

SOC=50% looks identical whether price is $20/MWh or $200/MWh. The agent cannot condition behavior on price.

State = (SOC, price, hour, …)

Now the agent can distinguish cheap charging opportunities from expensive discharge opportunities.

KEY IDEA **If relevant information is missing from the state, no algorithm can learn to use it.**

Real-world example: predictive maintenance

Balancing production today against failure risk tomorrow

STATE

Temperature
Vibration
Load
Health / RUL estimate

ACTION

Continue
Reduce load
Inspect
Maintain

OBJECTIVE

Production value
– energy cost
– maintenance cost
– failure consequences

KEY IDEA **Maintenance illustrates why maximizing immediate reward can be a poor strategy.**

Predictive maintenance in Python

A simplified transition function

```
def machine_step(state, action):
    T, vib, health = state

    if action == "CONTINUE":
        T += 3; vib += 0.3; health -= 0.03
        reward = 10
    elif action == "REDUCE_LOAD":
        T -= 4; vib -= 0.4
        reward = 4
    else:  # MAINTAIN
        T, vib = 60, 2.0
        health = min(1.0, health + 0.25)
        reward = -15

    return (T, vib, health), reward
```

Teaching point

CONTINUE looks attractive immediately (+10), while MAINTAIN costs −15. Long-term value must capture avoided degradation and failure.

KEY IDEA A reward model should reflect long-run operational objectives, not just short-term output.

Model-based vs model-free RL

Two ways to use experience

Model-based

Learn or use a model of transitions/rewards, then plan with it. Can be data-efficient but model errors matter.

Model-free

Learn values or policies directly from experience. Q-learning is the classical example.

KEY IDEA **“Model-free” means no explicit transition model is required for the learned decision rule.**

Why tabular RL stops scaling

Real states are too numerous or continuous

Battery discretization

10 SOC × 20 price × 20 demand × 20 solar × 24 hours = 1,920,000 states. With 3 actions: 5,760,000 Q-values.

Robotics / vision

Images, continuous sensors, positions and velocities create enormous state spaces. Exact tables become impractical.

KEY IDEA **Large state spaces motivate function approximation.**

Deep Q-Networks (DQN)

Approximate Q-values with a neural network



Core idea

Instead of storing one Q-value for every state–action pair, learn parameters θ that generalize across similar states. DQN combines Q-learning with neural-network approximation and stabilizing techniques such as replay buffers and target networks.

KEY IDEA **Deep RL changes the representation of Q—not the basic idea of long-term action value.**

REINFORCEMENT LEARNING

Deep Q-Networks (DQN)

Using deep neural networks to approximate the action-value function

Goal

Learn an optimal policy in high-dimensional state spaces by approximating $Q(s, a)$ with a deep neural network, using **experience replay** and a **target network** for stable training.

1 Why Deep Q-Networks?

Tabular Q-learning does not scale to large state spaces.

Tabular Q-learning

- Works for small, discrete state spaces
- Requires a table of size $|S| \times |A|$
- Not feasible for images, continuous states, or large environments

DQN

- Uses a neural network to approximate $Q(s, a)$
- Can handle high-dimensional inputs (e.g., images)
- Learns directly from raw observations (e.g., Atari games, robotics, control)

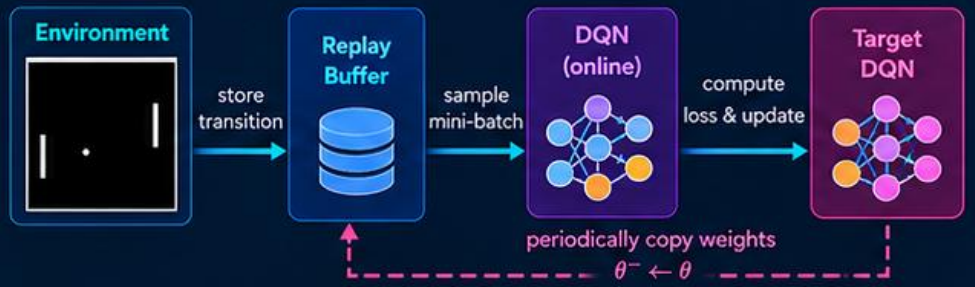
2 DQN architecture

A convolutional neural network maps the state to Q-values for all actions.



3 Overall training pipeline

Stabilization techniques make DQN work in practice.



4 Loss function (Q-learning target)

Minimize the difference between predicted Q-value and the target Q-value.

$$\mathcal{L}(\theta) = \left(r + \gamma \max_{a'} Q(s', a'; \theta^-) - Q(s, a; \theta)\right)^2$$

- θ – parameters of the online network
- θ^- – parameters of the target network
- r – observed reward
- s' – next state
- $\max_{a'} Q(s', a'; \theta^-)$ – target using target network

5 Experience replay

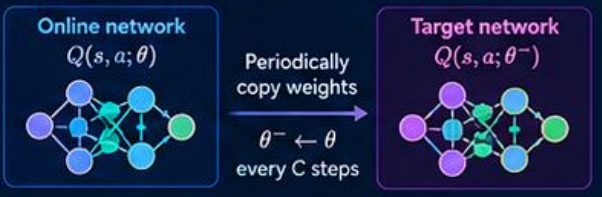
Breaks correlations between consecutive samples and improves data efficiency.



- Store transitions $(s, a, r, s', \text{done})$ in a buffer
- Sample random mini-batches for training
- Helps break temporal correlations and reuse data

6 Target network

A separate network provides stable targets.



- Target network parameters θ^- are a delayed copy of the online network parameters
- Reduces oscillations and divergence during training

7 DQN algorithm (simplified)

1. Initialize replay buffer $\mathcal{D}$, online network $Q(s, a; \theta)$, and target network $Q(s, a; \theta^-)$
2. For each episode:
   - Select action a using ε-greedy policy from $Q(s, a; \theta)$
   - Execute action, observe r, s', done
   - Store $(s, a, r, s', \text{done})$ in $\mathcal{D}$
   - Sample a random mini-batch from $\mathcal{D}$
   - Compute target: $y = r + \gamma \max_{a'} Q(s', a'; \theta^-)$ (or $y = r$ if done)
   - Update θ by minimizing $(y - Q(s, a; \theta))^2$
3. Periodically update target network: $\theta^- \leftarrow \theta$

8 Example: Atari Pong

DQN learns to play Pong from raw pixels.



9 Key takeaways

- DQN uses a neural network to approximate $Q(s, a)$.
- Experience replay breaks correlations and improves sample efficiency.
- A target network stabilizes training.
- DQN enables RL in high-dimensional state spaces (e.g., images, robotics, real-world control).
- Many extensions exist: Double DQN, Dueling DQN, Prioritized Replay, Rainbow, etc.

Policy-gradient methods

Learn the policy directly

$$\pi\theta(a \mid s)$$

A parameterized policy outputs action probabilities or continuous actions

Why useful

Natural for stochastic policies and continuous action spaces where enumerating all actions is difficult.

Learning signal

Adjust θ to increase the probability of actions associated with higher-than-expected return.

KEY IDEA **Value-based methods learn action values; policy-based methods optimize behavior directly.**

Actor–Critic

Combine a policy learner with a value estimator

ACTOR

Learns $\pi\theta(a|s)$
Chooses actions
Answers: “What should I do?”

CRITIC

Learns V(s), Q(s,a), or advantage
Evaluates outcomes
Answers: “How good was that?”

KEY IDEA The critic supplies a learning signal that helps improve the actor.

Safety and real deployment

A high reward is not permission to violate engineering constraints

- Safety-critical systems should encode hard operational limits separately from reward where appropriate.
- Training directly on physical equipment can be unsafe or expensive; simulators and offline data are often used first.
- Distribution shift, sensor faults, model uncertainty and human override must be considered.
- Reward misspecification can produce behavior that optimizes the number but not the real objective.

KEY IDEA **Real RL engineering is as much about problem formulation and validation as algorithm choice.**

End-to-end workflow for an RL project

From problem definition to monitored deployment



- Define constraints and success metrics before training.
- Check whether RL is actually needed; optimization or supervised learning may be simpler.
- Validate against strong baselines and test rare/unsafe conditions.

KEY IDEA **Good RL starts with a good decision problem—not with choosing an algorithm.**

One conceptual map

How the entire lecture fits together



Known model → Value / Policy Iteration

Unknown model → Experience → TD / Q-learning

Large state spaces → function approximation → DQN / policy gradients / Actor–Critic

KEY IDEA **Every advanced RL method still rests on states, actions, rewards, return and value.**

Final recap

Six ideas that should remain after the lecture

1. MDP

Formalizes sequential decisions under uncertainty.

2. Policy

Specifies behavior across states.

3. Value

Summarizes expected long-term return.

4. Bellman

Connects present value to reward + future value.

5. RL

Learns from experience when the model is unknown.

6. Q-learning

Uses TD error to learn action values model-free.

KEY IDEA **The central question: which action maximizes expected long-term return from the current state?**

Discussion questions

Use these to check conceptual understanding

- Why can a state with reward 0 still have high value?
- What information must be added if the current state is not Markov?
- Why does Q-learning use max Q(s′,a′) in its target?
- How do α, γ and ε affect learning in different ways?
- When would an MDP planning method be preferable to RL?
- What could go wrong with a poorly designed reward in a real industrial system?

KEY IDEA **Understanding the reasoning is more important than memorizing equations.**

THANK YOU

Questions • discussion • Python practice